

ESTUDIO — Carpeta controllers/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/controllers/` — **16 controllers**.

De qué va toda esta carpeta

Un **controller** es la **puerta de entrada** de la aplicación. Es la clase que recibe el pedido HTTP que manda el navegador y decide a quién pasárselo.

La regla de oro del proyecto es que **el controller es tonto a propósito**. Su trabajo se limita a tres cosas:

1. **Recibir** el pedido HTTP y convertir el JSON en un objeto Java (un DTO).
2. **Delegar** al service correspondiente, que es quien realmente piensa.
3. **Devolver** la respuesta con el código HTTP correcto.

No hay ni una regla de negocio adentro de un controller. Si te preguntan "¿por qué?", la respuesta es: porque así podés cambiar de HTTP a otra cosa (una app de escritorio, una cola de mensajes) sin tocar ni una línea de lógica. La lógica vive en `services/`.

Las convenciones que se repiten en TODOS los controllers

Antes de ir uno por uno, conviene entender el patrón, porque después se repite 16 veces:

Anotación / elemento	Qué significa en criollo
<code>@RestController</code>	"Esta clase atiende pedidos HTTP y lo que devuelve es JSON, no una página HTML."
<code>@RequestMapping("/api/v1/x")</code>	El prefijo de ruta de toda la clase. Si un método dice <code>/perfil</code> , la ruta completa es <code>/api/v1/x/perfil</code> .
<code>@RequiredArgsConstructor</code>	Anotación de Lombok. Le genera el constructor con los campos <code>final</code> , para que Spring pueda inyectar los services solos.
<code>@GetMapping</code> , <code>@PostMapping</code> , <code>@PutMapping</code> , <code>@PatchMapping</code> , <code>@DeleteMapping</code>	El verbo HTTP que atiende ese método.
<code>@PathVariable</code>	Un dato que viene en la URL . Ejemplo: en <code>/productos/5</code> , el <code>5</code> es el <code>id</code> .
<code>@RequestParam</code>	Un dato que viene después del ? . Ejemplo: <code>/localidades?provinciaId=94</code> .
<code>@RequestBody</code>	El JSON que viene en el cuerpo del pedido. Spring lo convierte solo en un DTO.
<code>@Valid</code>	"Antes de entrar al método, chequeá todas las validaciones del DTO." Si alguna falla → 400 automático.
<code>@AuthenticationPrincipal</code> <code>AuthenticatedUser</code> <code>usuario</code>	Muy importante . Le inyecta al método los datos del usuario logueado, sacados del JWT.
<code>ResponseEntity<ApiResponse<X>></code>	El tipo que devuelve siempre: un código HTTP + un cuerpo con <code>{mensaje, data}</code> .

Por qué `@AuthenticationPrincipal` es clave para la seguridad

Fijate que **ningún endpoint** recibe el `clienteId` o el `comercioId` por URL. Siempre lo sacan del token con `usuario.userId()`.

Si el endpoint fuera `GET /pedidos/cliente/{clienteId}`, cualquiera podría cambiar el número y ver los pedidos de otro. Al sacarlo del JWT (que está firmado y no se puede modificar), **es imposible hacerse pasar por otro usuario**. Eso es lo que evita una vulnerabilidad que se llama IDOR (Insecure Direct Object Reference), y es una respuesta muy buena si te preguntan por seguridad.

Por qué siempre `ApiResponse<T>`

Es una regla del proyecto (CLAUDE.md §3.1): **toda** respuesta de la API tiene el mismo formato:

```
{ "mensaje": "Pedido confirmado correctamente", "data": { ... } }
```

Ventaja: el frontend siempre sabe cómo leer la respuesta, sea éxito o error. Nunca hay que adivinar la forma del JSON según el endpoint.

LOS 16 CONTROLLERS

1. AuthController — `/api/v1/auth`

Para qué sirve

Todo lo que tiene que ver con **entrar y salir del sistema**: registro, login, logout, verificar la cuenta, recuperar contraseña, reactivar cuenta. Es el controller más grande (13 endpoints) porque concentra todo el ciclo de vida de una cuenta.

Usa **3 services**: `AuthService`, `RegistroService` y `CloudinaryService`.

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	POST /api/v1/auth/registro/cliente	Registra un cliente nuevo.	RegistroClienteRequestDTO (validado)	201 + UsuarioResponseDTO , mensaje "Cliente registrado correctamente, verificá tu email"	RegistroService.registrarCliente
2	POST /api/v1/auth/registro/cliente/foto-firma	Da la firma para que el cliente suba su foto de perfil durante el registro (antes de tener cuenta).	Nada	200 + CloudinarySignatureResponseDTO	CloudinaryService.generarFirmaFoto
3	POST /api/v1/auth/registro/comercio	Registra un comercio nuevo.	RegistroComercioRequestDTO	201 + UsuarioResponseDTO	RegistroService.registrarComercio
4	POST /api/v1/auth/registro/comercio/foto-firma	Igual que el 2, pero para la foto del comercio.	Nada	200 + CloudinarySignatureResponseDTO	CloudinaryService.generarFirmaFoto
5	POST /api/v1/auth/login	Inicia sesión.	LoginRequestDTO + el HttpServletRequest (para sacar IP y navegador)	200 + LoginResponseDTO (con el token)	AuthService.login
6	GET /api/v1/auth/verificar/{token}	Verifica la cuenta con un link (mecanismo viejo).	El token en la URL	200, sin data	AuthService.verificarEmail
7	POST /api/v1/auth/verificar	Verifica la cuenta con un código de 6 dígitos (mecanismo actual).	VerificarCodigoRequestDTO	200, sin data	AuthService.verificarEmailConCodigo
8	POST /api/v1/auth/reenviar-verificacion	Reenvía el código de verificación.	ReenviarVerificacionRequestDTO	200, mensaje deliberadamente ambiguo	AuthService.reenviarVerificacion
9	POST /api/v1/auth/recuperar-password	Pide el código para resetear la contraseña.	RecuperacionPasswordRequestDTO	200, mensaje ambiguo	AuthService.solicitarRecuperacionF
10	POST /api/v1/auth/recuperar-password/validar-codigo	Chequea que el código sea correcto, sin cambiar nada todavía.	ValidarCodigoRecuperacionRequestDTO	200, "Código válido"	AuthService.validarCodigoRecuperac
11	POST /api/v1/auth/recuperar-password/confirmar	Cambia la contraseña de verdad.	ConfirmarRecuperacionPasswordRequestDTO	200	AuthService.confirmarRecuperacionF
12	POST /api/v1/auth/reactivar-cuenta	Pide el código para reactivar una cuenta inactiva.	ReactivacionCuentaRequestDTO	200, mensaje ambiguo	AuthService.solicitarReactivacionC
13	POST /api/v1/auth/reactivar-cuenta/confirmar	Reactiva la cuenta.	ConfirmarReactivacionCuentaRequestDTO	200	AuthService.confirmarReactivacionC
14	POST /api/v1/auth/cambiar-password	Cambia la contraseña estando logueado.	CambioPasswordPerfilRequestDTO + el usuario del token	200	AuthService.cambiarPasswordDesdePe
15	POST /api/v1/auth/logout	Cierra la sesión.	Solo el usuario del token	200	AuthService.logout(usuario.sesionI

Tres detalles que suman puntos en la mesa

1. Los mensajes ambiguos son a propósito. Los endpoints 8, 9 y 12 responden "*Si existe una cuenta asociada a ese email, vas a recibir un código*". Nunca dicen "ese email no existe". Eso se llama evitar **enumeración de usuarios**: si el sistema respondiera distinto según exista o no, cualquiera podría probar mails y armar una lista de quién está registrado. Devolviendo siempre lo mismo, no se filtra nada.
2. Recuperar contraseña tiene 3 pasos, no 2. Pedir código → validar código → confirmar la contraseña nueva. El paso del medio existe puramente por experiencia de usuario: te avisa que el código está mal antes de que escribas la contraseña nueva dos veces, en vez de hacerte completar todo el formulario para recién ahí decirte que el código era incorrecto.
3. El logout usa `sessionId` , no `userId` . Cierra esa sesión puntual, no todas las del usuario. Eso es lo que hace que el logout sea real y no simbólico.

2. AdministradorController — /api/v1/administrador

Para qué sirve

El panel del administrador de la plataforma. Su función principal es **aprobar o rechazar los comercios** que se registran, más algunos listados de solo lectura.

Ojo: las categorías y los tags **no** están acá, tienen sus propios controllers.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/administrador/comercios/pendientes	Lista los comercios esperando aprobación.	Nada	200 + List<ComercioAdminResponseDTO>	listarComerciosPendientes
2	GET /api/v1/administrador/comercios	Lista los comercios ya aprobados.	Nada	200 + List<ComercioAdminResponseDTO>	listarComerciosAprobados
3	GET /api/v1/administrador/clientes	Lista todos los clientes (solo lectura).	Nada	200 + List<ClienteAdminResponseDTO>	listarClientes
4	GET /api/v1/administrador/perfil	Trae el nombre del admin logueado (para el header).	Usuario del token	200 + AdministradorResponseDTO	obtenerPerfil
5	GET /api/v1/administrador/metricas	Los números del dashboard.	Nada	200 + MetricasAdminResponseDTO	obtenerMetricas
6	PUT /api/v1/administrador/comercios/{comercioId}/resolver	Aprueba o rechaza un comercio.	comercioId en la URL + AprobacionComercioRequestDTO + admin del token	200, sin data	resolverAprobacion

Detalles

- El administrador no filtra por "sus" comercios. A diferencia del cliente (que ve solo sus pedidos) o del dueño (que ve solo sus productos), el admin ve **todo el sistema**. Es una consulta global por diseño.
- La ruta es `/resolver` , no `/resolucion` . La guía original decía `resolucion` , pero se dejó `resolver` para ser consistente con el resto de las acciones del proyecto (`/reactivar` , `/aceptar` , `/rechazar`), todas en infinitivo. Está documentado en CLAUDE.md §7bis como decisión de diseño, no como error.
- Un solo endpoint para aprobar y rechazar. El DTO adentro lleva la decisión (y el motivo si es un rechazo).

3. CarritoController — /api/v1/carrito

Para qué sirve

El carrito de compras del cliente. Rol **CLIENTE** exclusivamente.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/carrito	Muestra el carrito actual.	Usuario del token	200 + CarritoResponseDTO	verCarrito
2	POST /api/v1/carrito/items	Agrega un producto.	ItemCarritoRequestDTO	201 + CarritoResponseDTO	agregarItem
3	PUT /api/v1/carrito/items/{id}	Cambia la cantidad de un ítem.	id en URL + ActualizarCantidadItemCarritoRequestDTO	200 + CarritoResponseDTO	actualizarCantidad
4	DELETE /api/v1/carrito/items/{id}	Saca un producto del carrito.	id en URL	200 + CarritoResponseDTO	eliminarItem
5	DELETE /api/v1/carrito	Vacía el carrito entero.	Usuario del token	200, sin data	vaciarCarrito

Detalle de diseño lindo de contar

Casi todos devuelven el carrito completo actualizado, no solo el ítem que tocaste. Es a propósito: el frontend hace **una sola llamada** y ya tiene todo para repintar la pantalla (los ítems, el subtotal, el total). Si devolviera solo el ítem, el frontend tendría que hacer un segundo GET para recalcular el total. Menos llamadas, menos chance de que la pantalla quede desincronizada.

4. CatalogoController — /api/v1/catalogo

Para qué sirve

La vidriera pública. Es el único controller (junto con geografía y health) al que se puede entrar **sin estar logueado**. Cualquiera puede navegar comercios y productos antes de registrarse.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/catalogo/comercios	Lista los comercios aprobados .	Nada	200 + List<ComercioPublicoResponseDTO>	listarComerciosAprobados
2	GET /api/v1/catalogo/comercios/{id}/productos	Los productos de un comercio, con filtros opcionales.	id en URL + ? categoriaId= y ?tagId= opcionales	200 + List<ProductoResponseDTO>	listarProductosDelComercio
3	GET /api/v1/catalogo/productos	Búsqueda global de productos, paginada.	?categoriaId= , ?tagIds= (lista), ?q= (texto), ? pagina= — todos opcionales	200 + ProductosPaginadosResponseDTO	listarProductosGlobal
4	GET /api/v1/catalogo/filtros	Devuelve qué categorías y tags hay disponibles para filtrar.	Nada	200 + FiltrosCatalogoResponseDTO	listarFiltrosDisponibles

Dos cosas importantes

1. ComercioPublicoResponseDTO es un DTO aparte, y ese es el punto. El comercio tiene un "representante legal" con nombre y DNI. Ese dato **no puede viajar sin autenticación**. Por eso hay tres DTOs distintos para lo mismo:

DTO	Quién lo usa	Lleva el DNI del representante
ComercioPublicoResponseDTO	Catálogo público	No
ComercioResponseDTO	El comercio viendo su propio perfil	Sí
ComercioAdminResponseDTO	El administrador	Sí, y más datos

Es un ejemplo perfecto de **para qué sirven los DTOs**: el mismo dato de la base, expuesto con distinto nivel de detalle según quién pregunta. Si el controller devolviera la entidad Comercio directamente, no habría forma de esconder el DNI.

2. El endpoint 3 devuelve ProductosPaginadosResponseDTO , no una lista pelada. Porque además de los productos necesita decir cuántas páginas hay en total.

5. CategoriaController — /api/v1/categorias

Para qué sirve

CRUD de categorías de producto (Pizzas, Hamburguesas, Bebidas...). Las gestiona **solo el administrador**, pero **cualquiera autenticado puede leerlas**.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	POST /api/v1/categorias	Crea una categoría.	CategoriaRequestDTO	201 + CategoriaResponseDTO	crear
2	PUT /api/v1/categorias/{id}	Edita una.	id + CategoriaRequestDTO	200 + CategoriaResponseDTO	editar
3	GET /api/v1/categorias	Lista todas.	Nada	200 + List<CategoriaResponseDTO>	listar
4	DELETE /api/v1/categorias/{id}	Da de baja.	id	200, sin data	baja
5	PUT /api/v1/categorias/{id}/reactivar	Vuelve a activarla.	id	200 + CategoriaResponseDTO	reactivar

El concepto de "baja lógica" — buena pregunta de mesa

El DELETE **no borra la fila de la base**. Pone `activo = false`. Por eso existe `/reactivar`: si borrara de verdad, no habría nada que reactivar.

Por qué se hace así: si borrás una categoría que tiene 40 productos asociados, esos productos quedarían apuntando a una categoría que ya no existe — se rompe la integridad referencial. Con baja lógica, la categoría desaparece de los listados nuevos pero los datos históricos siguen coherentes. Se llama **soft delete**.

Sobre el GET siendo público para cualquier autenticado: hizo falta porque el dueño del comercio necesita leer las categorías para llenar el selector al crear un producto. Se resolvió en `SecurityConfig` poniendo una regla específica de GET **antes** de la genérica de ADMINISTRADOR (ver ESTUDIO-CONFIG.md).

6. TagController — /api/v1/tags

Para qué sirve

Exactamente el mismo patrón que categorías, pero con **tags** (etiquetas libres: "sin TACC", "vegano", "picante"...).

Métodos

#	Endpoint	Qué hace	Devuelve	Service
1	POST /api/v1/tags	Crea un tag.	201 + TagResponseDTO	crear
2	PUT /api/v1/tags/{id}	Edita.	200 + TagResponseDTO	editar
3	GET /api/v1/tags	Lista todos.	200 + List<TagResponseDTO>	listar
4	DELETE /api/v1/tags/{id}	Baja lógica.	200, sin data	baja
5	PUT /api/v1/tags/{id}/reactivar	Reactiva.	200 + TagResponseDTO	reactivar

Diferencia conceptual Categoría vs Tag

	Categoría	Tag
Cuántas por producto	Una sola	Varias
Relación	Producto → Categoria (muchos a uno)	Producto ↔ Tag (muchos a muchos, vía la tabla ProductoTag)
Para qué	Clasificar (a qué grupo pertenece)	Describir (qué características tiene)

Una milanesa **es** de la categoría "Carnes", y **tiene** los tags "sin TACC" y "abundante".

7. ClienteController — /api/v1/clientes

Para qué sirve

Que el cliente vea y edite **su propio perfil**. Rol CLIENTE.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/clientes/perfil	Trae los datos del cliente logueado.	Usuario del token	200 + ClienteResponseDTO	verPerfil
2	PUT /api/v1/clientes/perfil	Edita el perfil.	ClienteEditarPerfilRequestDTO	200 + ClienteResponseDTO	editarPerfil

Detalle importante

La ruta es `/perfil`, **no** `/clientes/{id}`. Otra vez: el id sale del token, no de la URL, así que es imposible ver o editar el perfil ajeno.

Además, el DTO de edición **solo tiene 3 campos** (`nombre`, `apellido`, `telefono`). El email, el DNI y la fecha de nacimiento son de **solo lectura** — no se pueden cambiar. Y no porque el service los ignore, sino porque **el DTO ni siquiera los tiene**. Si mandás un `email` en el JSON, Spring simplemente lo descarta al armar el objeto. Eso es una defensa a nivel de diseño, más fuerte que un `if`.

8. ComercioController — /api/v1/comercios

Para qué sirve

Que el dueño vea y edite el perfil de su comercio, incluida la foto. Rol **DUENO**.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/comercios/perfil	Perfil del comercio.	Usuario del token	200 + ComercioResponseDTO	verPerfil
2	PUT /api/v1/comercios/perfil	Edita el perfil.	ComercioPerfilRequestDTO	200 + ComercioResponseDTO	editarPerfil
3	POST /api/v1/comercios/perfil/foto/firma	Pide la firma para subir la foto a Cloudinary.	Usuario del token	200 + CloudinarySignatureResponseDTO	generarFirmaFotoPerfil
4	PUT /api/v1/comercios/perfil/foto	Guarda la URL de la foto ya subida.	FotoPerfilComercioRequestDTO	200 + ComercioResponseDTO	actualizarFotoPerfil

El patrón de subida de imágenes en 2 pasos (aparece en 3 controllers)

Vale la pena entenderlo bien porque se repite:

- Frontend: "Dame permiso para subir una foto" → POST ../firma
- Backend: devuelve una firma criptográfica temporal
- Frontend: sube el archivo DIRECTO a Cloudinary con esa firma
- Cloudinary: devuelve la URL de la imagen
- Frontend: "Guardá esta URL" → PUT ../foto
- Backend: valida que sea una URL de Cloudinary y la guarda

Por qué así y no mandar el archivo al backend: el servidor nunca maneja archivos pesados, no consume su ancho de banda, y no necesita disco. Y la firma es lo que evita que cualquiera suba lo que quiera a la cuenta de Cloudinary del proyecto: sin firma válida, Cloudinary rechaza la subida.

9. RedSocialController — /api/v1/comercios/redes-sociales

Para qué sirve

Que el comercio administre sus redes sociales (Instagram, Facebook, WhatsApp...). Es una de las funcionalidades **nuevas del proyecto completo**, que no existía en el MVP.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/comercios/redes-sociales	Lista las redes activas del comercio.	Usuario del token	200 + List<RedSocialResponseDTO>	listarActivas
2	POST /api/v1/comercios/redes-sociales	Agrega una red.	RedSocialRequestDTO	201 + RedSocialResponseDTO	agregar
3	PUT /api/v1/comercios/redes-sociales/{id}	Edita una.	id + RedSocialRequestDTO	200 + RedSocialResponseDTO	editar
4	DELETE /api/v1/comercios/redes-sociales/{id}	Baja lógica.	id	200, sin data	darDeBaja

Fijate que el método se llama `listarActivas` y el delete `darDeBaja` : otra vez baja lógica, mismo criterio que categorías y tags.

Nota sobre la ruta: cae dentro de `/api/v1/comercios/**`, así que `SecurityConfig` la protege con rol **DUENO** automáticamente, sin necesidad de una regla propia.

10. ProductoController — /api/v1/productos

Para qué sirve

El CRUD de productos del comercio **más toda la gestión de la galería de imágenes**. Es el controller con más endpoints después de Auth (9). Rol **DUENO**.

Métodos — grupo A: el producto en sí

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	POST /api/v1/productos	Crea un producto.	ProductoRequestDTO	201 + ProductoResponseDTO	crearProducto
2	PUT /api/v1/productos/{id}	Edita un producto.	id + ProductoRequestDTO	200 + ProductoResponseDTO	editarProducto
3	GET /api/v1/productos	Lista los productos del comercio logueado.	Usuario del token	200 + List<ProductoResponseDTO>	listarProductosDelComercio
4	PATCH /api/v1/productos/{id}/estado	Cambia el estado (disponible / agotado / discontinuado).	id + CambioEstadoProductoRequestDTO	200 + ProductoResponseDTO	cambiarEstado

Métodos — grupo B: la galería de imágenes

#	Endpoint	Qué hace	Recibe	Devuelve	Service
5	POST /api/v1/productos/{id}/cloudinary/firma	Firma para subir una imagen nueva.	id	200 + CloudinarySignatureResponseDTO	generarFirmaImagen
6	POST /api/v1/productos/{id}/imagenes	Registra la imagen ya subida.	id + ImagenProductoRequestDTO	201 + ImagenProductoResponseDTO	agregarImagen
7	DELETE /api/v1/productos/{id}/imagenes/{imagenId}	Borra una imagen.	2 ids en URL	200, sin data	eliminarImagen
8	PATCH /api/v1/productos/{id}/imagenes/{imagenId}/orden	Reordena una imagen en la galería.	2 ids + OrdenImagenRequestDTO	200 + ImagenProductoResponseDTO	reordenarImagen
9	POST /api/v1/productos/{id}/imagenes/{imagenId}/recorte/firma	Firma para subir una versión recortada de una imagen existente.	2 ids	200 + CloudinarySignatureResponseDTO	generarFirmaRecorteImagen
10	PATCH /api/v1/productos/{id}/imagenes/{imagenId}/url	Reemplaza la URL de una imagen (después del recorte).	2 ids + UrlImagenRequestDTO	200 + ImagenProductoResponseDTO	actualizarUrlImagen

Cosas para tener a mano

- **Límite de 5 imágenes por producto.** Si intentás la sexta, el service devuelve **409 Conflict**. Está probado de verdad.
- **Por qué PATCH y no PUT en los 3 casos que lo usan:** PUT significa "reemplazá el recurso entero", PATCH significa "cambiá solo este campo". Cambiar el estado, el orden o la URL de una imagen es un cambio parcial, así que corresponde PATCH. El resto del proyecto usa PUT porque reemplaza el objeto completo.
- **Los endpoints 9 y 10 son del editor de recorte.** El usuario recorta la foto en el navegador, sube la versión recortada a Cloudinary con la firma del 9, y con el 10 le dice al backend "ahora la imagen es esta otra URL".

11. PedidoController — /api/v1/pedidos

Para qué sirve

Los pedidos, **de los dos lados**: el cliente que los hace y el comercio que los recibe. La ruta se divide en /cliente y /comercio, y eso no es cosmético — es lo que le permite a SecurityConfig separar los permisos por rol solo mirando el prefijo.

Métodos — lado CLIENTE

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	POST /api/v1/pedidos/cliente	Confirma el pedido: convierte el carrito en un pedido.	PedidoRequestDTO	201 + PedidoResponseDTO	confirmarPedido
2	GET /api/v1/pedidos/cliente	El historial de pedidos del cliente.	Usuario del token	200 + List<PedidoResponseDTO>	listarPedidosCliente

Métodos — lado COMERCIO (rol DUENO)

#	Endpoint	Qué hace	Recibe	Devuelve	Service
3	GET /api/v1/pedidos/comercio	Los pedidos que le llegaron al comercio.	Usuario del token	200 + List<PedidoResponseDTO>	listarPedidosComercio
4	GET /api/v1/pedidos/comercio/resumen-hoy	Métricas del día para el dashboard.	Usuario del token	200 + ResumenPedidosHoyResponseDTO	obtenerResumenHoy
5	PUT /api/v1/pedidos/comercio/{id}/aceptar	Acepta un pedido.	id	200 + PedidoResponseDTO	aceptarPedido
6	PUT /api/v1/pedidos/comercio/{id}/rechazar	Rechaza un pedido, con motivo obligatorio.	id + RechazoPedidoRequestDTO	200 + PedidoResponseDTO	rechazarPedido

Dos decisiones de diseño para saber explicar

1. **Aceptar y rechazar son dos endpoints, no uno con un flag.** La guía original planteaba un solo `PATCH /pedidos/{id}/resolucion` con un booleano `aceptar` adentro. Se hicieron dos porque la intención queda explícita en la URL: leyendo el log del servidor ya sabés qué pasó, sin tener que abrir el cuerpo del pedido. Y es `PUT` porque el estado del pedido se reemplaza por un valor conocido, no se parchea.

2. **No hay un endpoint de "detalle de pedido por id".** El frontend, cuando querés ver un pedido, llama a `GET /pedidos/cliente` y filtra del lado del navegador. Puede sonar poco elegante, pero tiene una ventaja de seguridad: como el listado ya viene filtrado por el JWT, **es imposible que te devuelva un pedido ajeno**. Un endpoint `GET /pedidos/{id}` tendría que chequear a mano que ese pedido sea tuyo. *(Está documentado como decisión consciente en los mapeos de la Fase 16.)*

12. NotificacionController — /api/v1/notificaciones

Para qué sirve

Las notificaciones dentro de la app (no por mail): "tenés un pedido nuevo", "tu pedido fue aceptado", "tu comercio fue aprobado".

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/notificaciones	Lista las notificaciones del usuario.	Usuario del token	200 + List<NotificacionResponseDTO>	listar
2	PUT /api/v1/notificaciones/{id}/leida	Marca una como leída.	id	200 + NotificacionResponseDTO	marcarLeida
3	GET /api/v1/notificaciones/no-leidas/contador	Cuántas sin leer hay (para el numerito de la campanita).	Usuario del token	200 + un Long	contarNoLeidas

Cómo funciona la "notificación en tiempo real" — pregunta probable

No es en tiempo real de verdad. Se usa **polling**: el frontend le pregunta al backend *¿cuántas no leídas tengo?* **cada 15 segundos**, con un `setInterval` de JavaScript.

	Polling (lo que se usó)	WebSockets (la alternativa)
Cómo funciona	El cliente pregunta cada X segundos	El servidor avisa al cliente cuando pasa algo
Complejidad	Muy baja	Alta
Latencia	Hasta 15 segundos de demora	Instantáneo
Carga	Una consulta por usuario cada 15s	Una conexión abierta por usuario

Para este proyecto polling alcanza: la escala es chica y 15 segundos de demora en enterarte de un pedido no rompe nada. Es una decisión de "lo simple que funciona" vs "lo sofisticado que no hacía falta" — y decirlo así en la mesa muestra criterio.

El endpoint 3 devuelve **solo un número**, no la lista entera. Es el que se llama cada 15 segundos, así que se hizo lo más liviano posible; la lista completa solo se trae cuando el usuario abre la campanita.

13. GeografiaController — /api/v1/geografia

Para qué sirve

Alimentar los selectores de **Provincia** y **Localidad** de los formularios de dirección. Público, sin login (lo necesitás durante el registro, cuando todavía no tenés cuenta).

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/geografia/provincias	Las 24 provincias argentinas.	Nada	200 + List<ProvinciaResponseDTO>	listarProvincias
2	GET /api/v1/geografia/localidades?provinciaId=X	Las localidades de esa provincia.	provinciaId como query param obligatorio	200 + List<LocalidadResponseDTO>	listarLocalidadesPorProvincia

Detalles

- **Es un selector dependiente:** elegís provincia, y recién ahí se cargan sus localidades. Por eso el `provinciaId` es obligatorio — hay más de 4000 localidades en total, traerlas todas de una sería absurdo.
- **De dónde salieron los datos:** de un ETL (Extract-Transform-Load) que corrió una sola vez contra la **API Georef** del gobierno argentino y cargó 24 provincias y ~4038 localidades. El script es re-ejecutable sin duplicar (usa `ON DUPLICATE KEY UPDATE`).
- **Acá se encontró un bug real:** si no mandabas el `provinciaId` , Spring tiraba `MissingServletRequestParameterException` , que debía dar **400...** pero daba **401**. La causa era que `/error` no estaba en las rutas públicas de `SecurityConfig` (explicado en ESTUDIO-CONFIG.md).

14. UsuarioController — /api/v1/usuarios

Para qué sirve

La foto de perfil del usuario (la persona), distinta de la foto del comercio. Es funcionalidad del proyecto completo.

Métodos

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	POST /api/v1/usuarios/{id}/foto-perfil/firma	Firma para subir la foto.	id en URL + usuario del token	200 + CloudinarySignatureResponseDTO	generarFirmaFotoPerfil(id, usuario.userId())
2	PATCH /api/v1/usuarios/{id}/foto-perfil	Guarda la URL de la foto.	id + FotoPerfilUsuarioRequestDTO	200 + UsuarioResponseDTO	actualizarFotoPerfil
3	DELETE /api/v1/usuarios/{id}/foto-perfil	Saca la foto de perfil.	id	200 + UsuarioResponseDTO	eliminarFotoPerfil

La excepción a la regla — y por qué no es un agujero

Este es el único controller que recibe un `{id}` en la URL además del usuario del token. Rompe el patrón de todos los demás.

Pero fijate que le pasa los dos al service: `generarFirmaFotoPerfil(id, usuario.userId())` . O sea, el service compara: *"¿el id que me pediste es el mismo del token?"*. Si no coinciden, corta. Es la misma protección de siempre, solo que hecha explícitamente en el service en vez de implícitamente por no tener el parámetro.

15. HealthController — /api/v1/health

Para qué sirve

Un endpoint mínimo para preguntar "¿el backend está vivo?". Público.

#	Endpoint	Qué hace	Devuelve
1	GET /api/v1/health	Responde que está operativo.	200 + {"mensaje": "Bajoneá backend operativo", "data": null}

Es el único controller sin ningún service — no llama a nadie, no toca la base. Se hizo en la Fase 3 para probar que el arranque de Spring y el formato `ApiResponse` funcionaban antes de tener entidades. En un despliegue real, este es el endpoint que un balanceador de carga consulta para saber si el servidor puede recibir tráfico.

16. TestController — /api/v1/test

Para qué sirve

Endpoints auxiliares exclusivos del entorno de pruebas. Sirven para que los tests automatizados (Postman, Playwright) puedan obtener un token de verificación sin tener que leer un mail de verdad.

#	Endpoint	Qué hace	Recibe	Devuelve	Service
1	GET /api/v1/test/token-verificacion?email=X	Devuelve el token de verificación pendiente de ese usuario.	email	200 + el token	obtenerTokenVerificacionPendiente
2	GET /api/v1/test/token?email=X&tipo=Y	Versión general: cualquier tipo de token.	email + tipo (TipoToken)	200 + el token	obtenerTokenPendiente

Por qué esto NO es un agujero de seguridad (importante saberlo explicar)

Suena horrible: un endpoint público que te da tokens de cualquier usuario. Pero fíjate en la anotación de la clase: `@Profile("test")` .

Eso significa que Spring **solo crea esta clase si la aplicación arranca con el perfil `test` activo**. En el perfil normal o en producción, el bean nunca se crea, la ruta nunca se registra, y un pedido a `/api/v1/test/token` devuelve **404** — porque literalmente no hay nadie que responda.

La **protección es estructural, no una lista de permisos**. El hecho de que la ruta figure en `RUTAS_PUBLICAS` de `SecurityConfig` no la hace alcanzable: permitir el paso a una puerta que no existe no abre nada. Es una respuesta muy sólida si te lo cuestionan en la mesa.

Tabla resumen de los 16 controllers

Controller	Ruta base	Rol requerido	Endpoints
AuthController	/api/v1/auth	Público (salvo cambiar-password y logout)	15
AdministradorController	/api/v1/administrador	ADMINISTRADOR	6
CategoriaController	/api/v1/categorias	ADMIN (GET: cualquier autenticado)	5
TagController	/api/v1/tags	ADMIN (GET: cualquier autenticado)	5
CatalogoController	/api/v1/catalogo	Público	4
GeografiaController	/api/v1/geografia	Público	2
HealthController	/api/v1/health	Público	1
ClienteController	/api/v1/clientes	CLIENTE	2
CarritoController	/api/v1/carrito	CLIENTE	5
ComercioController	/api/v1/comercios	DUENO	4
RedSocialController	/api/v1/comercios/redes-sociales	DUENO	4
ProductoController	/api/v1/productos	DUENO	10
PedidoController	/api/v1/pedidos	CLIENTE en /cliente , DUENO en /comercio	6
NotificacionController	/api/v1/notificaciones	Autenticado	3
UsuarioController	/api/v1/usuarios	Autenticado	3
TestController	/api/v1/test	Solo perfil <code>test</code>	2

Los códigos HTTP que usa el proyecto, y cuándo

Código	Cuándo se usa acá	Ejemplo
200 OK	Lecturas y actualizaciones.	GET /carrito , PUT /productos/5
201 Created	Cuando se crea algo nuevo .	POST /productos , POST /pedidos/cliente
400 Bad Request	Una validación de DTO falló. Lo tira <code>@Valid</code> solo.	Mandar un email sin <code>@</code>
401 Unauthorized	Sin token o token inválido.	Pegarle a /carrito sin loguearte
403 Forbidden	Rol insuficiente.	Un CLIENTE en /administrador/**
404 Not Found	El recurso no existe.	PUT /productos/9999
409 Conflict	Choca contra una regla de negocio.	Email ya registrado, 6ª imagen de un producto, producto de otro comercio en el carrito
429 Too Many Requests	Rate limit del registro.	Más de 5 firmas por minuto

Nota: el proyecto **evita el 204 No Content** a propósito. Aun en los DELETE devuelve **200 con un mensaje** , para que el frontend siempre tenga algo que mostrarle al usuario.

Preguntas típicas de mesa

"¿Qué hace un controller en tu proyecto?" Recibe el pedido HTTP, lo convierte a un DTO, se lo pasa al service y devuelve la respuesta con el código HTTP correcto. Nada más — no tiene lógica de negocio.

"¿Cómo sabés qué usuario está haciendo el pedido?" Con `@AuthenticationPrincipal authenticatedUser` . Spring me inyecta los datos que el filtro JWT sacó del token. Nunca los recibo por URL, justamente para que no se puedan falsear.

"¿Por qué devolvés siempre `ApiResponse` ?" Para que toda la API tenga el mismo formato de respuesta, sea éxito o error. El frontend siempre sabe cómo leerla.

"¿Cómo validás lo que te mandan?" Con `@Valid` sobre el `@RequestBody` . Las reglas están declaradas como anotaciones en el DTO. Si algo falla, Spring corta antes de entrar al método y el `GlobalExceptionHandler` devuelve un 400 con el detalle de qué campo estuvo mal.

"¿Cómo subís las imágenes?" En dos pasos: el frontend pide una firma al backend, sube el archivo directo a Cloudinary con esa firma, y después le manda al backend solo la URL. El backend nunca toca el archivo.

Índice de archivos cubiertos en este documento

Archivo	Ruta base	En una línea
<code>AuthController.java</code>	<code>/api/v1/auth</code>	Registro, login, logout, verificación, recuperación de contraseña.
<code>AdministradorController.java</code>	<code>/api/v1/administrador</code>	Aprobación de comercios, listados y métricas del admin.
<code>CarritoController.java</code>	<code>/api/v1/carrito</code>	Ver, agregar, actualizar, eliminar y vaciar el carrito.
<code>CatalogoController.java</code>	<code>/api/v1/catalogo</code>	Vidriera pública de comercios y productos, con filtros.
<code>CategoriaController.java</code>	<code>/api/v1/categorias</code>	CRUD de categorías con baja lógica.
<code>TagController.java</code>	<code>/api/v1/tags</code>	CRUD de tags con baja lógica.
<code>ClienteController.java</code>	<code>/api/v1/clientes</code>	Ver y editar el perfil propio del cliente.
<code>ComercioController.java</code>	<code>/api/v1/comercios</code>	Perfil del comercio y su foto.
<code>RedSocialController.java</code>	<code>/api/v1/comercios/redes-sociales</code>	Redes sociales del comercio.
<code>ProductoController.java</code>	<code>/api/v1/productos</code>	CRUD de productos y galería de imágenes.
<code>PedidoController.java</code>	<code>/api/v1/pedidos</code>	Pedidos del lado cliente y del lado comercio.
<code>NotificacionController.java</code>	<code>/api/v1/notificaciones</code>	Notificaciones in-app con polling.
<code>GeografiaController.java</code>	<code>/api/v1/geografia</code>	Provincias y localidades para los selectores.
<code>UsuarioController.java</code>	<code>/api/v1/usuarios</code>	Foto de perfil del usuario.
<code>HealthController.java</code>	<code>/api/v1/health</code>	Chequeo de que el backend está vivo.
<code>TestController.java</code>	<code>/api/v1/test</code>	Tokens de prueba, solo bajo perfil <code>test</code> .